

CSE3023

WEB-BASED APPLICATION DEVELOPMENT

Servlet (Part 2)

 Assoc. Prof. Ts. Dr. Wan Nural Jawahir Hj Wan Yussof
| Faculty of Computer Science & Mathematics



Learning Outcomes

Understanding the Servlet Concept

Dynamic Content Handler

Servlets serve as the backbone for creating dynamic, interactive server-side web content.

Request-Response Intermediary

Acts as the middle layer between a user's browser and the web server's logic.



Creating and Using Servlets

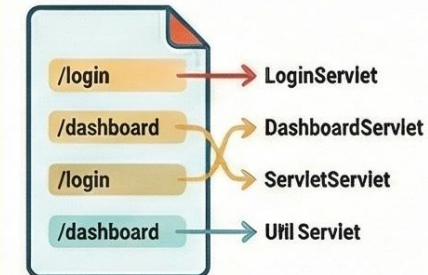
Class Extension

Create a Java class that extends standard servlet libraries to inherit web capabilities.



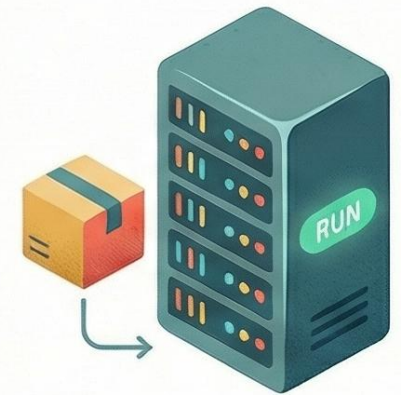
Configuration and Mapping

Define specific URL patterns to determine which requests the servlet should handle.



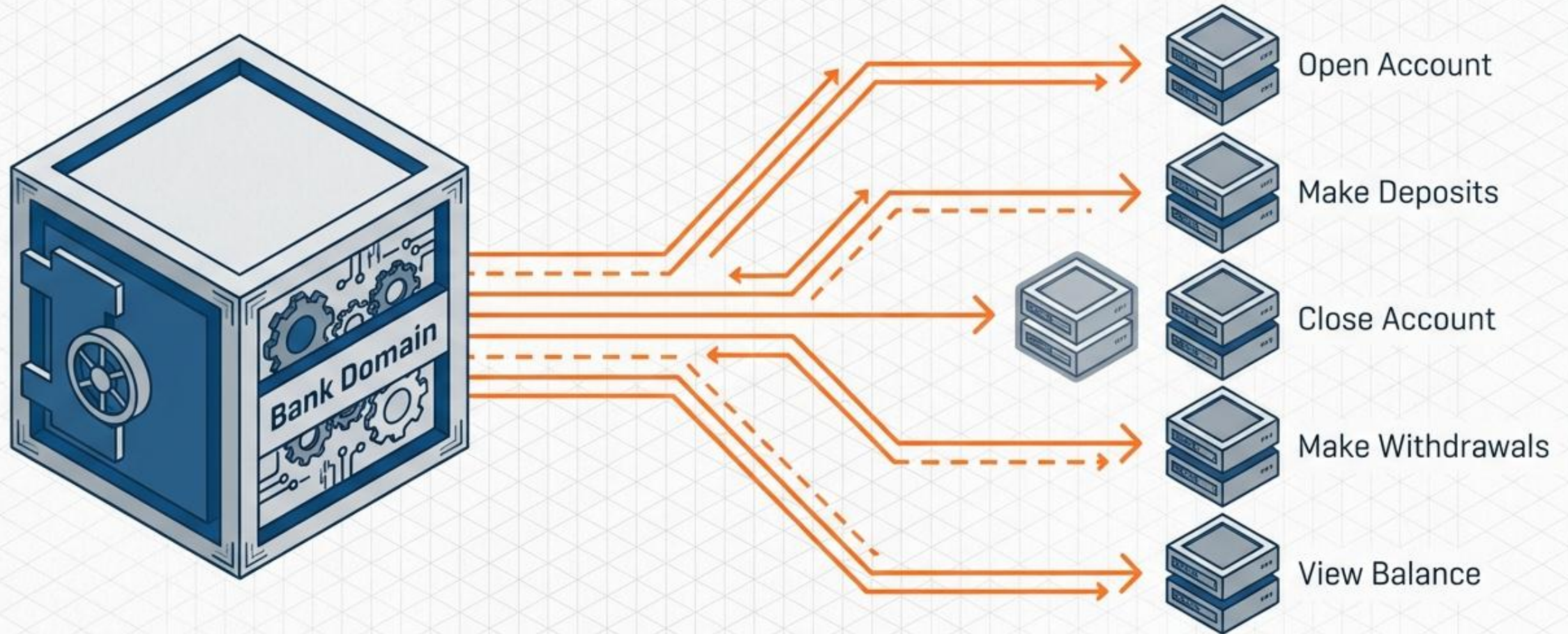
Deployment

Package and run the servlet within a web container or application server.



Real-world business processes demand modular architecture

Enterprise applications cannot be managed as single, monolithic scripts. They are divided into distinct, purpose-built operational tasks.



Isolated components require active coordination

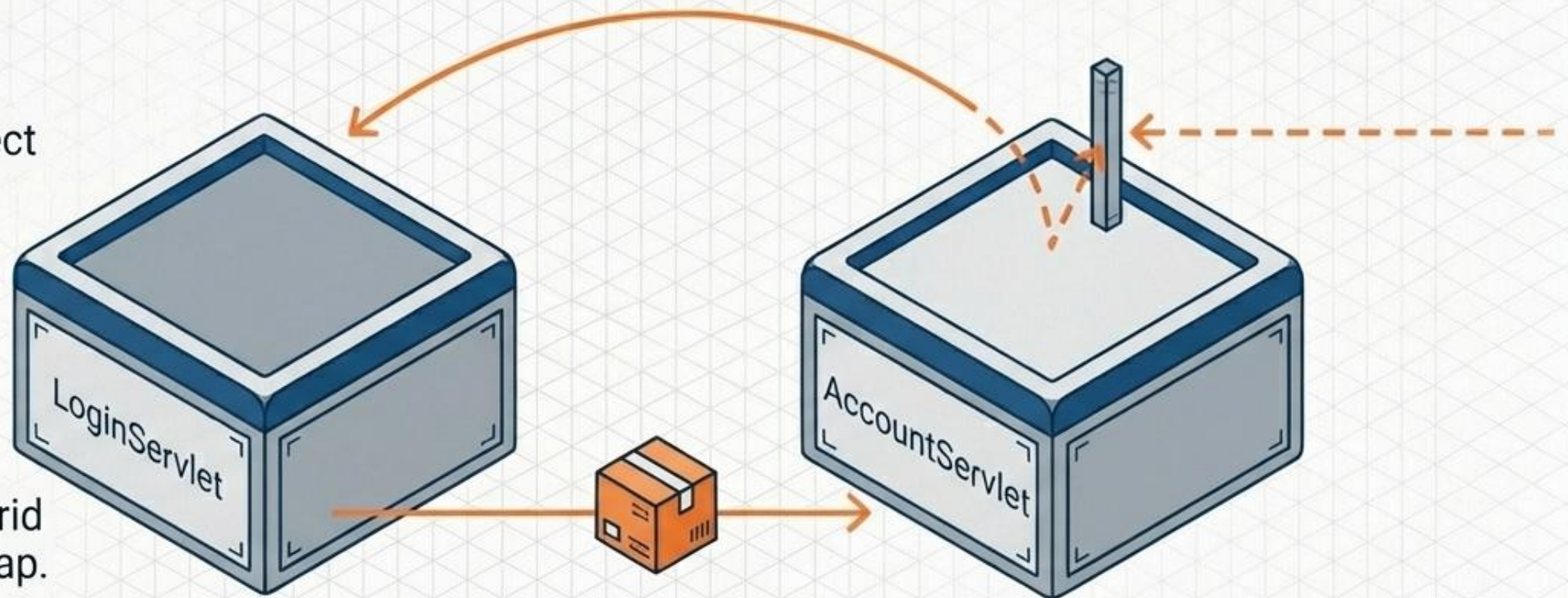
To manage banking processes, we isolate responsibilities into two distinct servlets: a LoginServlet for authentication and an AccountServlet for business logic. This physical separation creates two critical coordination scenarios.

Top Path:

Scenario 1: Redirect unauthenticated traffic back to origin.

Bottom Path:

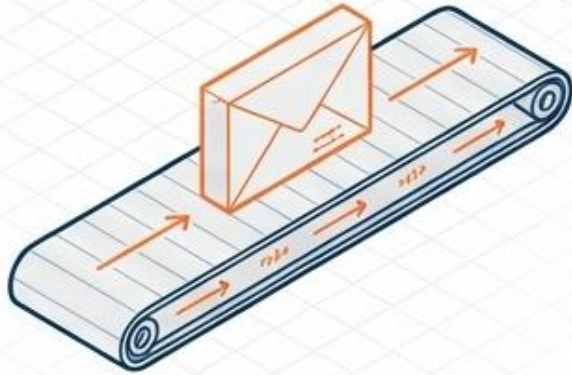
Scenario 2: Share authenticated userid state across the gap.



Three distinct containers manage data payloads

Servlets overcome isolation by placing data payloads into specialized scope objects. The choice of object determines how long the data survives and who can access it.

ServletRequest



Stores data strictly for the lifespan of a single HTTP request.

HttpSession



Stores user-specific session data that persists across multiple requests.

ServletContext



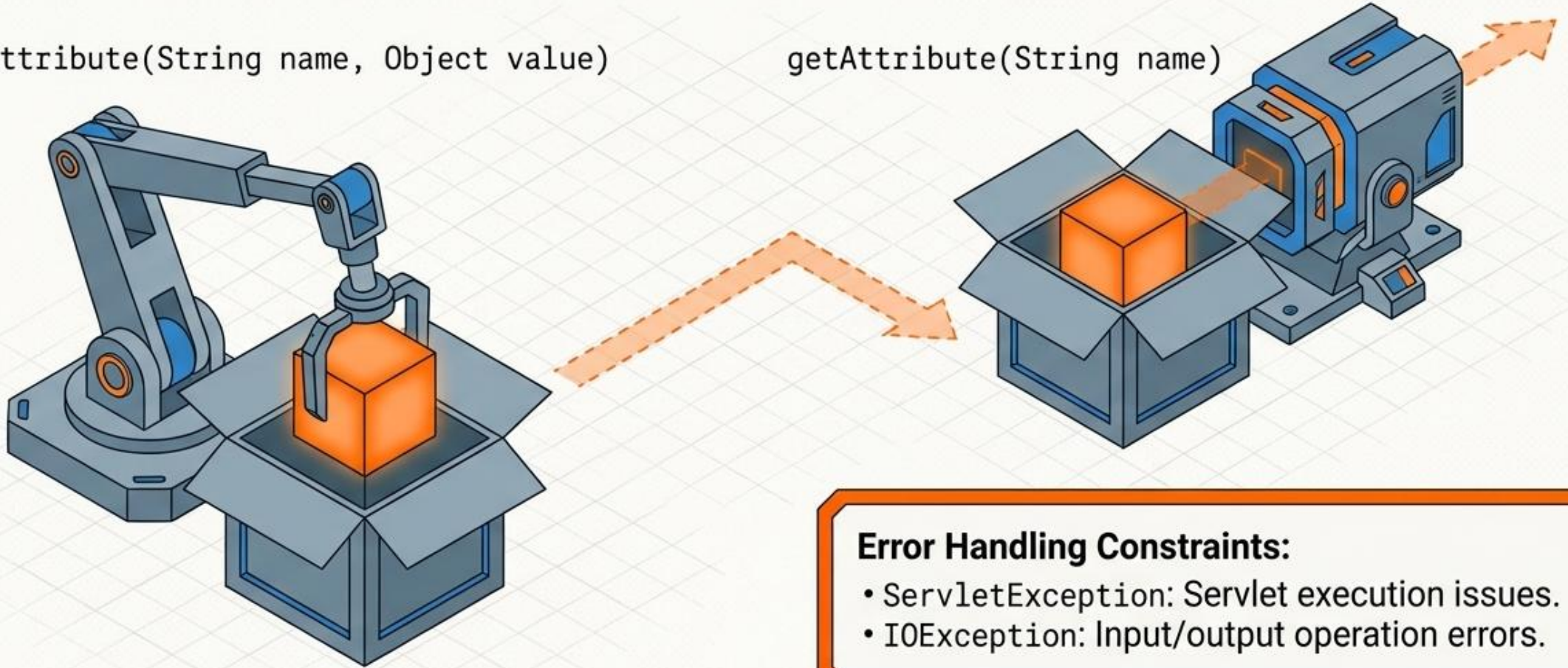
Shares global data accessible across the entire application ecosystem.

Standardized methods control cargo loading and retrieval

Regardless of the scope container used, developers utilize standardized methods to handle the data objects. Disruptions in this process trigger specific system exceptions.

`setAttribute(String name, Object value)`

`getAttribute(String name)`

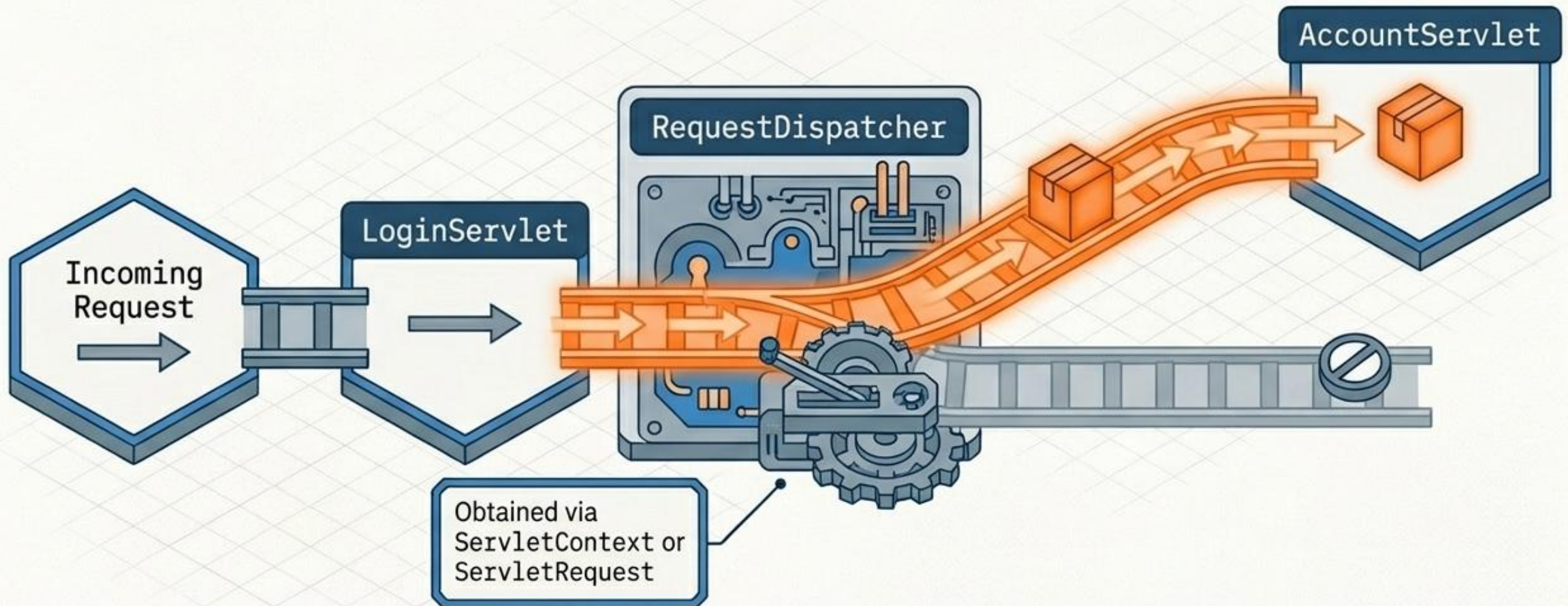


Error Handling Constraints:

- `ServletException`: Servlet execution issues.
- `IOException`: Input/output operation errors.

The RequestDispatcher acts as the internal traffic controller

When a user inputs their credentials, the LoginServlet does not generate the final response. Instead, it utilizes the RequestDispatcher to route the request directly to the AccountServlet.



Routing methods dictate the flow of system control

The `RequestDispatcher` offers two distinct methods for handling request transitions. The choice depends on whether the original servlet needs to retain control of the final response.

	<code>void forward()</code>	<code>void include()</code>
Primary Function	Process partially, then hand off for final response.	Merge contents of another resource into the current response.
Transfer of Control	Permanent handoff.	Temporary pass; calling resource resumes control to completion.
Header/Status Modification	Permitted (unless response is already committed).	Strictly forbidden (attempts to do so are ignored).
Critical Constraints	Throws <code>IllegalStateException</code> if called after response is committed.	Included resource cannot set the headers or status code.

Execution pathways: The Bullet vs. The Boomerang

The Bullet: `forward()`

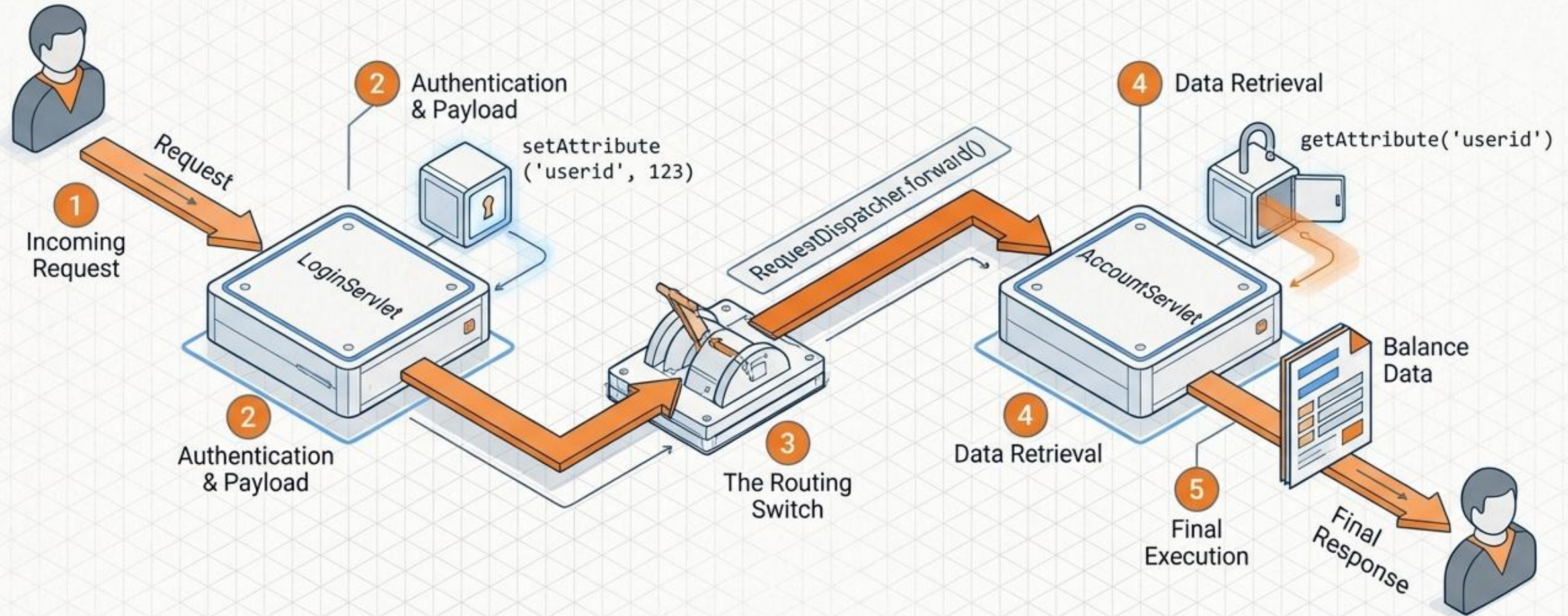


The Boomerang: `include()`

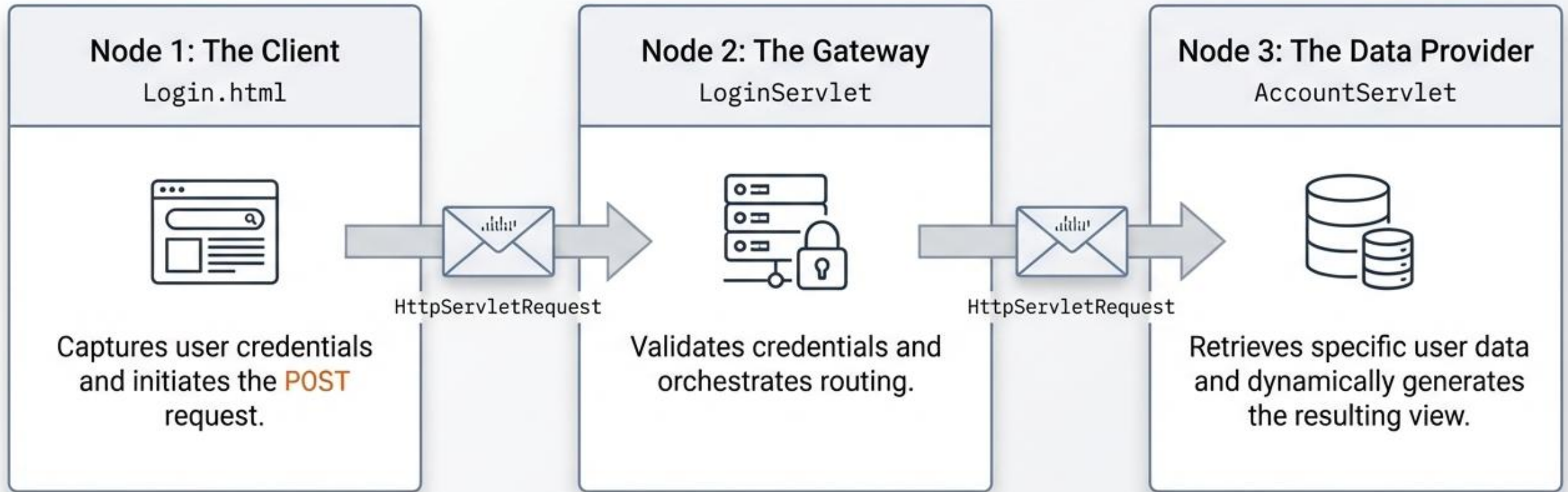


Synthesis: The complete multi-servlet lifecycle

The isolated components, shared state payloads, and routing mechanics working together to execute a single banking process.



The Three-Stage Architecture Model



The data travels sequentially from left to right, entirely orchestrated by backend handoffs.

Step 1: The Client Interface

```
16 <form name="login" id="login" method="POST" action="LoginServlet">
17     Username: <input type="text" name="userid" required autofocus >
18     </br>
19     </br>
20     Password: <input type="password" name="password" required>
21     </br>
22     </br>
23     <input type="submit" name="submit" value="Login">
24 </form>
```

The Target

action='LoginServlet' acts as the destination address for the form's payload.

The Protocol

method='POST' ensures the data is sent securely in the request body.

The Payload

The 'userid' and 'password' fields become the initial parameters packed into the HTTP request envelope.

Simulating the Database: The **init()** Method

LoginServlet: users Hashtable

Key (userid)	Value (password)
user1	admin
user2	admin
user3	admin

```
public void init() {  
    users.put("user1", "admin");  
    users.put("user2", "admin");  
    users.put("user3", "admin");  
}
```

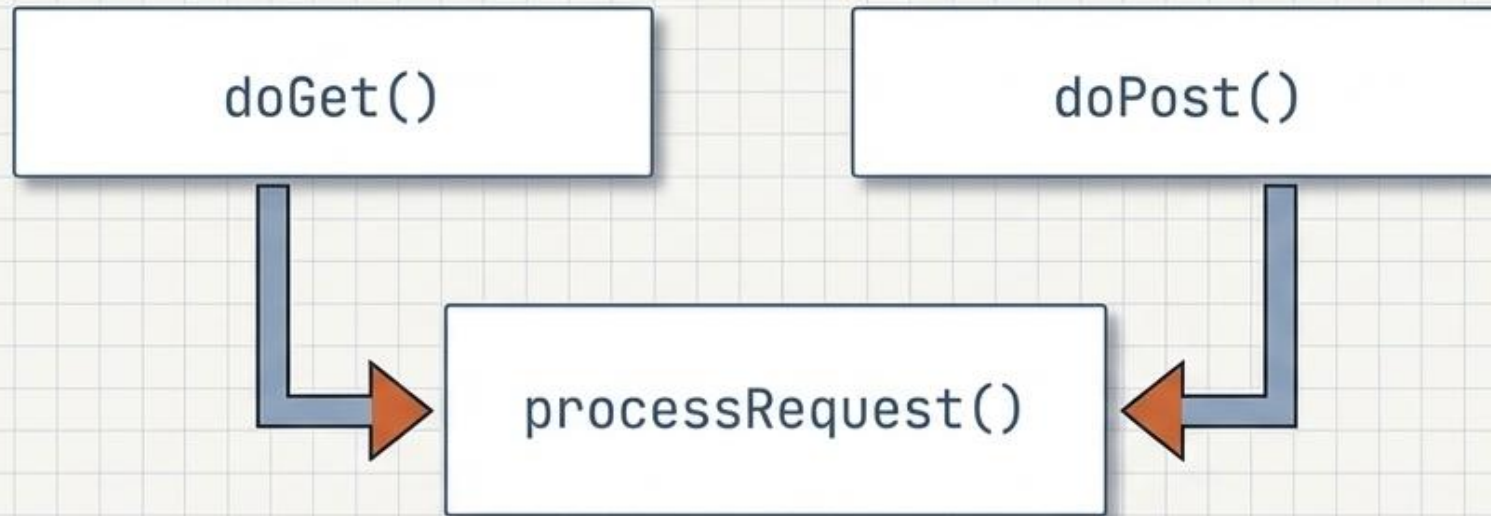
AccountServlet: data Hashtable

Key (userid)	Value (Transaction Array)
user1	11/02/2013 : 2000.00, 22/03/2013 : 800.00, 02/04/2013 : 500.00...
user2	10/01/2013 : 1300.00, 15/02/2013 : 550.00, 27/03/2013 : 400.00...
user3	09/02/2013 : 2500.00, 13/03/2013 : 1000.00, 10/04/2013 : 400.00...

```
public void init() {  
    data.put("user1", new TransactionArray(11/02/2013 : 2000.00,  
    data.put("user2", new TransactionArray(10/01/2013 : 1300.00,  
    data.put("user3", new TransactionArray(09/02/2013 : 2500.00,  
}
```

Takeaway: Both servlets use the `init()` method to instantiate an in-memory Hashtable at startup, creating an isolated mock data layer.

Step 2: The Gateway (LoginServlet)



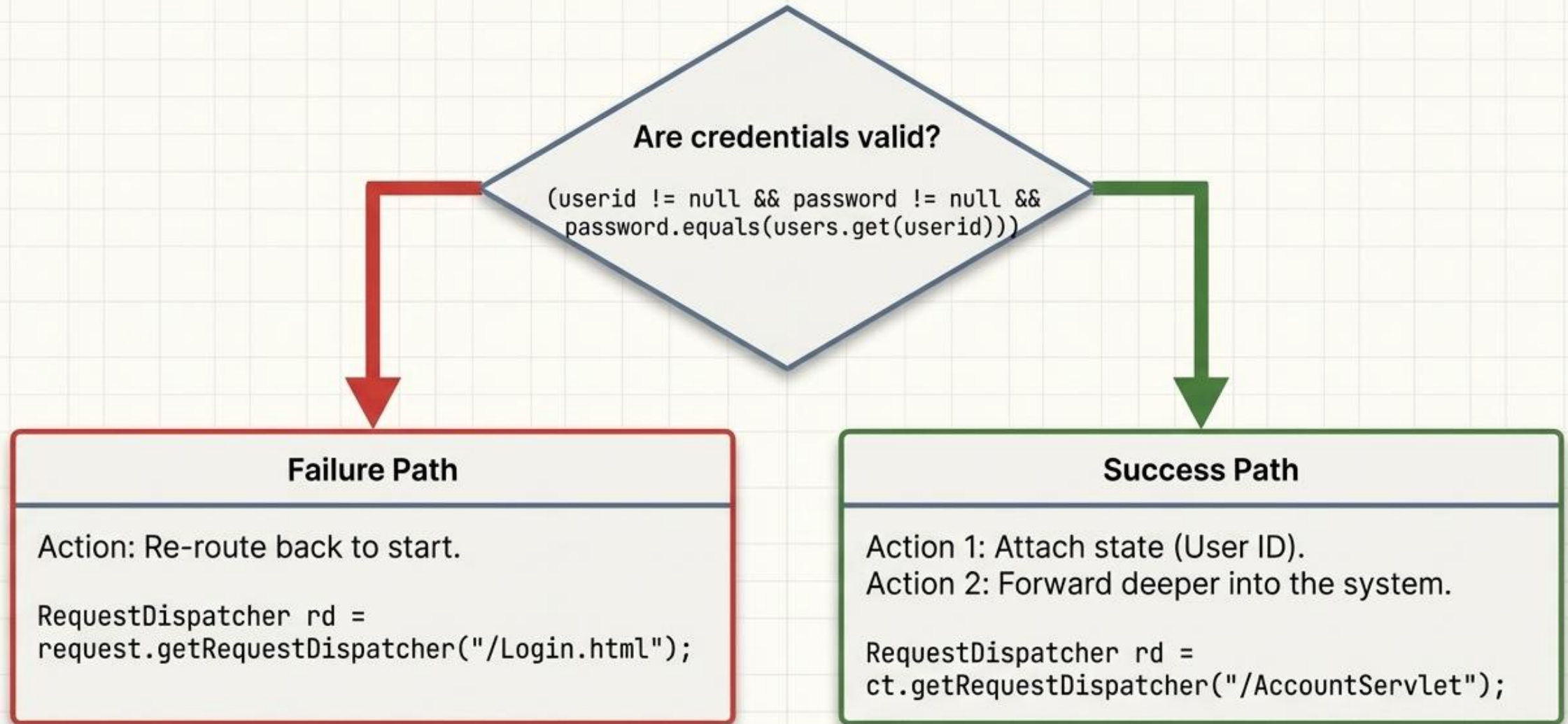
```
String userid = request.getParameter("userid");  
String password = request.getParameter("password");
```



Unpacking the Envelope

The servlet extracts the 'userid' and 'password' submitted by the HTML form from the **HttpServletRequest** object to begin validation.

Visualizing Authentication Logic



Deep Dive: The Art of Coordination

Anatomy of a Handoff

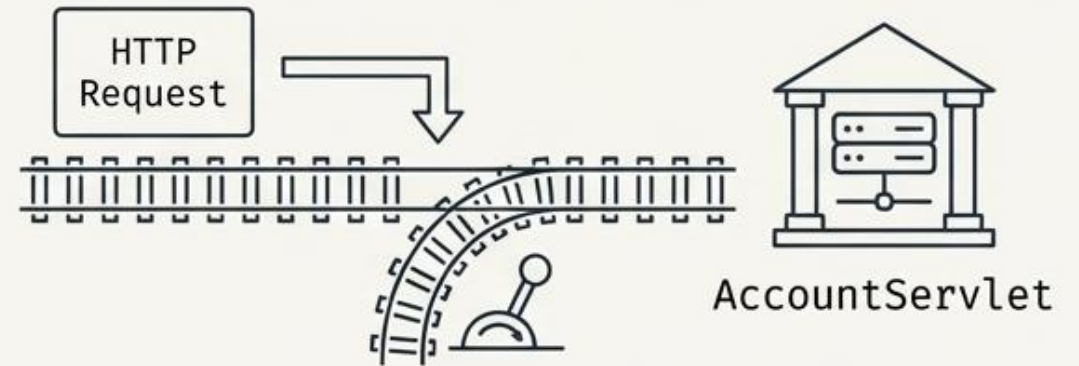
State Sharing (setAttribute)



```
request.setAttribute("userid", userid);
```

Data sharing is achieved by attaching temporary attributes to the request object before passing it along.

Control Forwarding (RequestDispatcher)

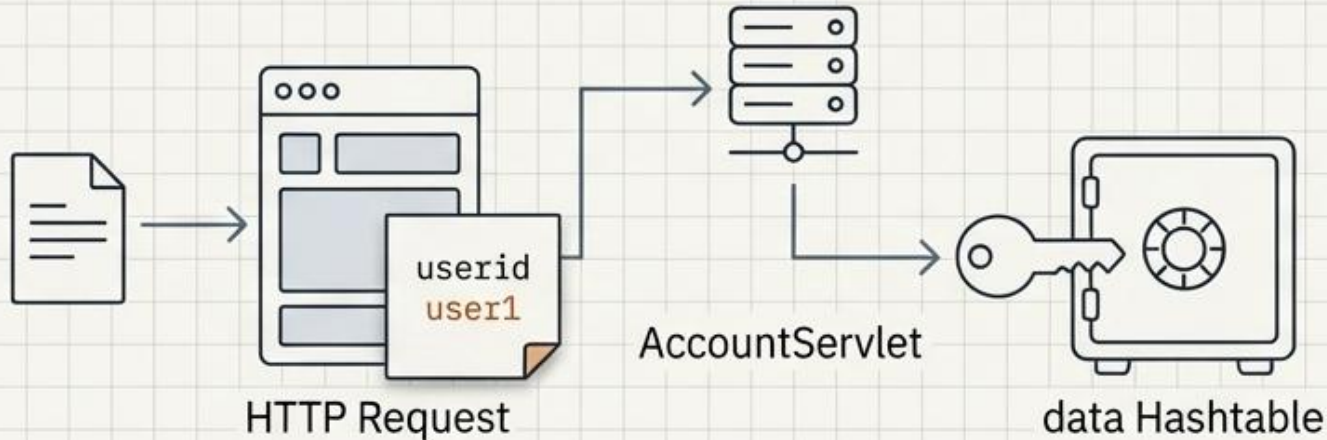


```
rd.forward(request, response);
```

The ServletContext acts as shared memory, allowing the RequestDispatcher to seamlessly hand off the client's request to another resource on the server without the browser knowing.

Step 3: The Data Provider: Catching the Request

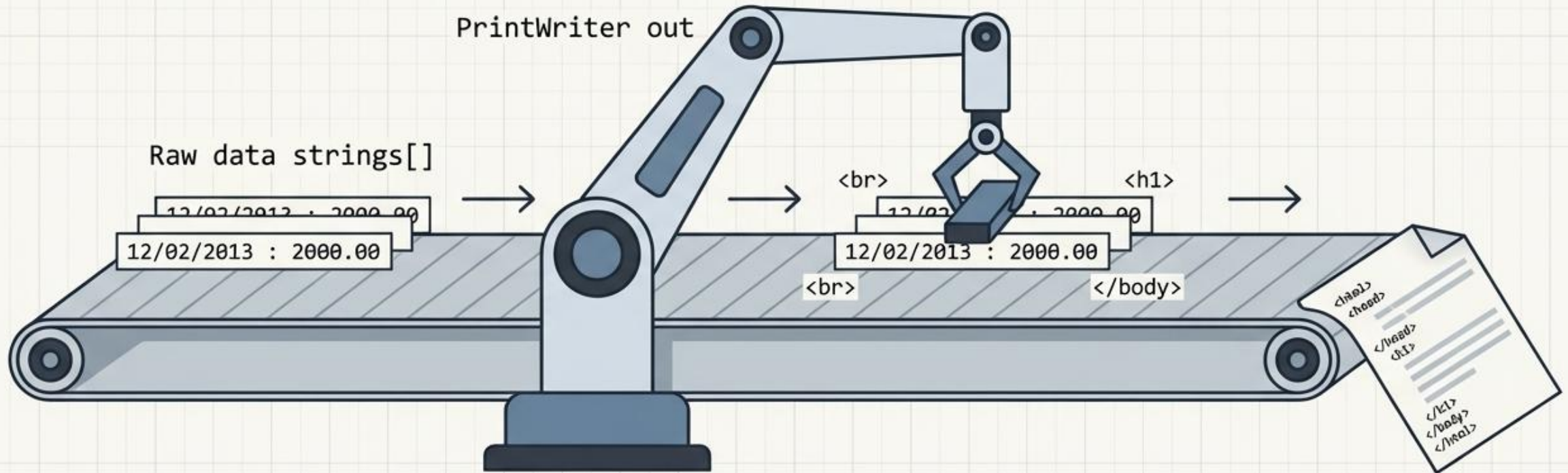
```
public class Servlet main(get setting, component) {  
  
    //Get userid set by LoginServlet..  
    String userid = (String) request.getAttribute("userid");  
    ...  
}
```



getParameter vs. getAttribute

getParameter:	Data submitted directly from the client's HTML form.
getAttribute:	Data internally passed from another servlet on the server.

Dynamic View Generation



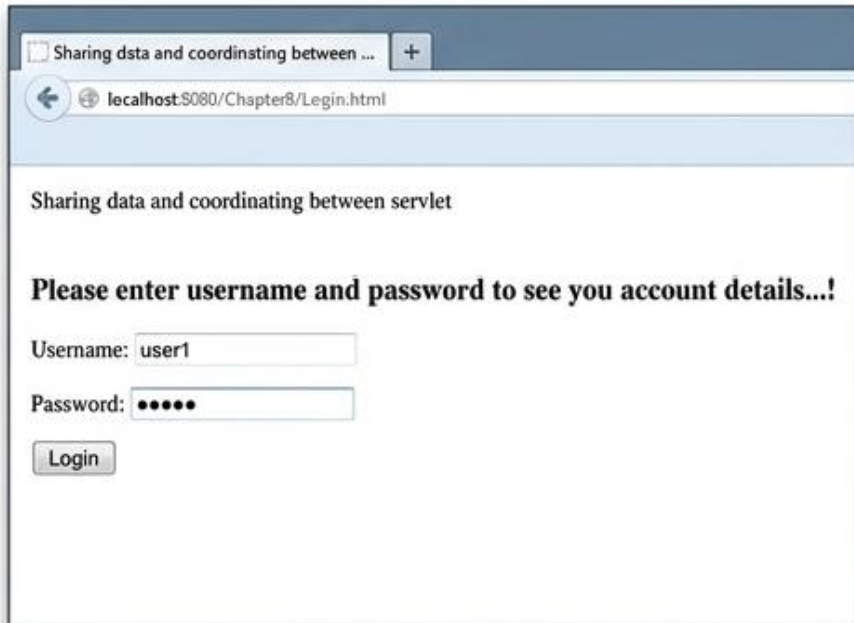
```
for (int i=0; i<records.length; i++) {  
    out.println(records[i] + "<br>");  
}
```

Dynamic View Generation

The servlet isn't serving a static file; it is writing the HTML response line-by-line based on the specific user's data.

The End-to-End User Experience

The Input



Sharing data and coordinating between ... +

localhost:5080/Chapter8/Login.html

Sharing data and coordinating between servlet

Please enter username and password to see you account details...!

Username:

Password:

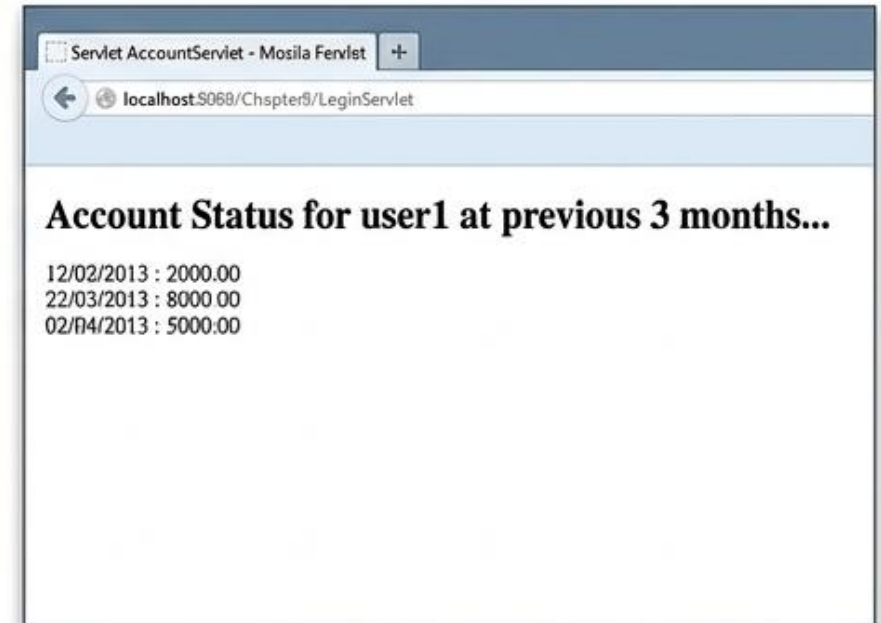
Login

Step 1: The user submits credentials via Login.html



Invisible
LoginServlet
Step 2

The Output



Servlet AccountServlet - Mosila Ferret +

localhost:5068/Chapter8/LoginServlet

Account Status for user1 at previous 3 months...

12/02/2013	: 2000.00
22/03/2013	: 8000.00
02/04/2013	: 5000.00

Step 3: The user views the dynamically constructed response from AccountServlet.

Component Synthesis Matrix

Separation of concerns across the scenario architecture.

Component	Primary Role	Key Input Mechanism	Key Output Mechanism
 Login.html	Client Interface	User keystrokes	HTTP POST parameters
 LoginServlet	Authentication & Routing	<code>request.getParameter()</code>	<code>RequestDispatcher.forward()</code>
 AccountServlet	Dynamic View Generation	<code>request.getAttribute()</code>	<code>PrintWriter.println()</code>

Core Principles: The Servlet API



Share Memory, Not URLs

Use `request.setAttribute()` to securely pass internal state (like verified User IDs) between servlets without exposing it to the client browser.



Consolidate Control Flow

Use `RequestDispatcher` via the `ServletContext` to seamlessly hand off processing duties, allowing for strict separation of authentication logic from data retrieval.



Dynamic Views on Demand

Use `PrintWriter` to iterate through server-side data structures and construct custom HTML responses on the fly based on the internal state passed through the pipeline.